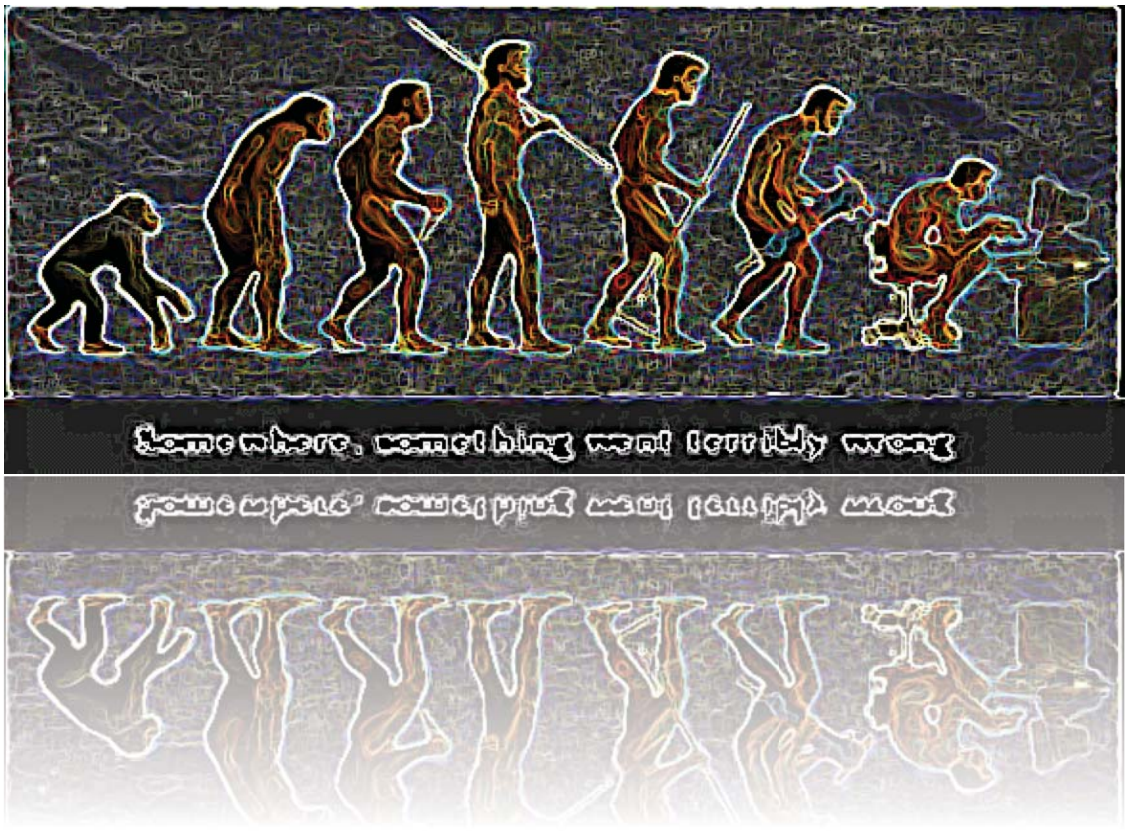
Computational Intelligence II: Evolutionary Optimisation

Chapter 2: Basic structure of Genetic Algorithms



Overview of Genetic Algorithms (GAs)

- ⌘ GAs is a class of search or optimisation techniques which mimics elements from natural genetics and the theory of evolution using a computational framework.

- ⌘ GAs are based on the Darwinian doctrine of the evolution of species through the mechanism of natural selection.

- ⌘ They use a simple memory-based structure, the ***population***, which contains a number of ***members*** or ***individuals***. Each individual has a dual representation:

  - ◎ Its ***genotype*** which contains the ***genome*** or ***chromosome*** needed to describe the individual at the low (coded) level. In GAs this material is usually simplified as a haploid (single) chromosome structures composed of ***genes***, each taking values, called ***alleles***, from a pre-specified set called the ***alphabet***.

  - ◎ Its ***phenotype***, the actual (visible) expression of its characteristics, such as the solution vector itself.

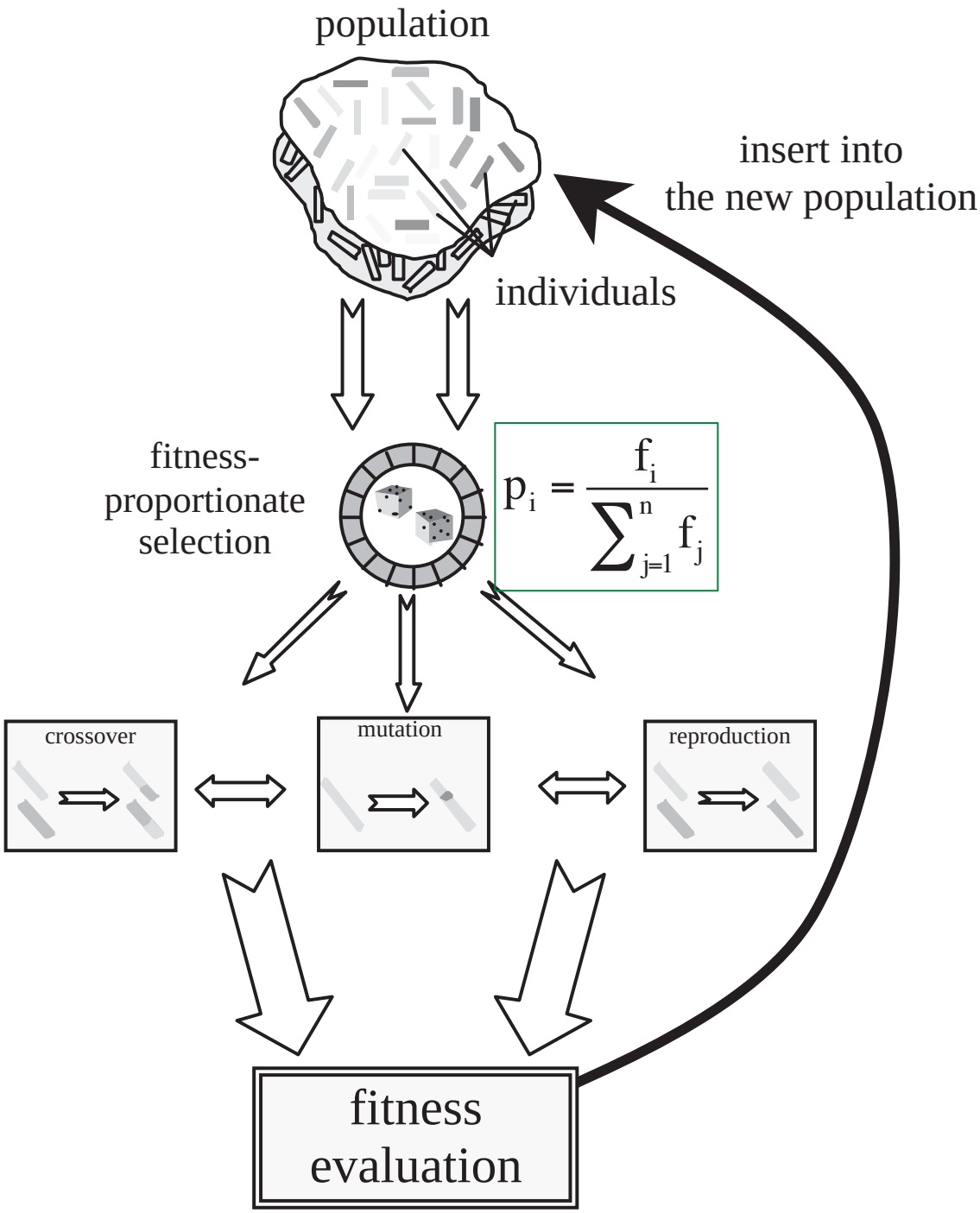
Main GA stages

- **Genetic coding:**
  - Each solution of the problem at hand is encoded as a chromosome. Each problem is different and every problem has more than one possible encodings (representations).
- **Population:**
  - A finite number of chromosomes to interact with each other. The number of strings should be large enough to have diversity (i.e., the samples are not taken from the same solution space region), but also small enough to have computational efficiency.
- **Evaluation of objective function:**
  - A direct measure of the problem function, in terms of quality and optimality.
- **Fitness value calculation:**
  - Conversion of the objective value to a fitness or probability of "goodness", for mating and survival reasons.
- **Reproduction:**
  - Preference for survival and mating is given to solutions with high fitness values / mating probabilities.
- **Crossover:**
  - Mix genetic material from parents to create new children solutions.
- **Mutation:**
  - Perturb the genes of the children to introduce diversity.

A typical GA algorithm

1. **Initialise** a random starting population.
2. **Repeat** until convergence is satisfied:
   a. Evaluate the **objective function F** for all the members in the current population.
   b. Calculate the **fitness values f** of all members.
   c. Perform **parent selection** based on their fitness probabilities.
   d. Apply **crossover** to selected parents to generate offspring.
   e. Apply **mutation** to the offspring.
   f. Replace some parents with the offspring and **create a new population** (generation).
3. Take the <u>top (best) individual</u>, decode it, and pass it to the user as the "final answer".

GA algorithm overview



Advantages & drawbacks of GAs

⌘ GAs exhibit certain **advantages** compared to other techniques:

- GAs are stochastic optimisers with the characteristic that they typically use some parameter encoding to work on and not always the parameters themselves.
- Also, their search does not involve a single point in the solution space, but a set of points processed simultaneously.
- Calculus-based techniques (gradient methods) are local in scope as they perform search in the vicinity of a current point. Also, problems with discontinuous, undifferentiable and multimodal objective functions make such methods unsuitable.
- Exhaustive techniques, such as the enumerative ones, random walks, tree (depth/breadth) search and ones using partial knowledge (such as hill climbing, best-first, A*) suffer from combinatorial problems when the search space is of very high dimensionality.
- GAs outperform many such techniques for the optimisation of a large number of problems. They can handle efficiently and robustly highly dimensional search spaces with discontinuous, multimodal and noisy objective functions.
- GAs have been applied to a multitude of diverse applications, such as NP-hard combinatorial problems, adaptive control, game-playing, machine learning, etc.

⌘ Continued...

- GAs base their search on directed stochastic decisions, where the repetitive application of the genetic operators and the fact that fitter solutions are granted more reproductive opportunities, leads to an overall improvement in the population. Fitter individuals disseminate their high quality schemata (solution parts or characteristics) to the future generations more frequently and the less fit ones are progressively doomed to extinction.

⌘ GAs have certain **drawbacks** that cannot be dismissed:

- First, being a stochastic optimisation technique can not always guarantee convergence. Although convergence can be proved subject to rigorous conditions, these conditions are hardly used in practice due to the available computational resources.
- Also, a GA can be time consuming, when too many objective function evaluations are required.
- Furthermore, it requires fine-tuning of many parameters which may affect performance.
- Finally, because maximal efficiency of a GA-based optimiser can most certainly be gained by developing problem-specific operators, the design/implementation phase is often cumbersome. The latter implies the unequivocal trade-off between the ability to have a generic all-purpose slower optimiser and an efficient one.

Genetic encoding

- Genetic coding permits the mapping of the phenotype (a solution) to the genotype.

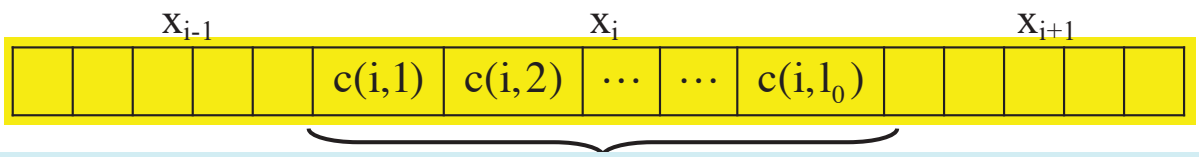
- Problems to be solved can be continuous, e.g. minimising a continuous function $f(x_1,x_2)$, or discrete, e.g., Travelling Salesman Problem, Digital control systems, etc.

- All the parameters that need to be optimised must be encoded in the chromosome structure.

- Precision & discretisation issues must be taken into account for continuous variables:
  - The function to be optimised is locally smooth. This allows other points to be constructed by interpolation.
  - The spacing of the mesh is sufficiently small that values of the cost function around the global optimum are closer to the true optimum than to any other local extremum.
  - E.g., if x in f(x) can be any number in [-100,+100], how many bits do you need to represent x in binary?

Binary chromosome coding

Binary encoding / decoding scheme, where m variables are encoded in an l-length chromosome, with l_0 bits per variable x_i, i=1,…,m, i.e., $l = m \times l_0$



i_{th} chromosome segment of length l_0, which corresponds to variable x_i

- We assume that each x_i is within a user-defined range $[x_{i,min}, x_{i,max}]$.

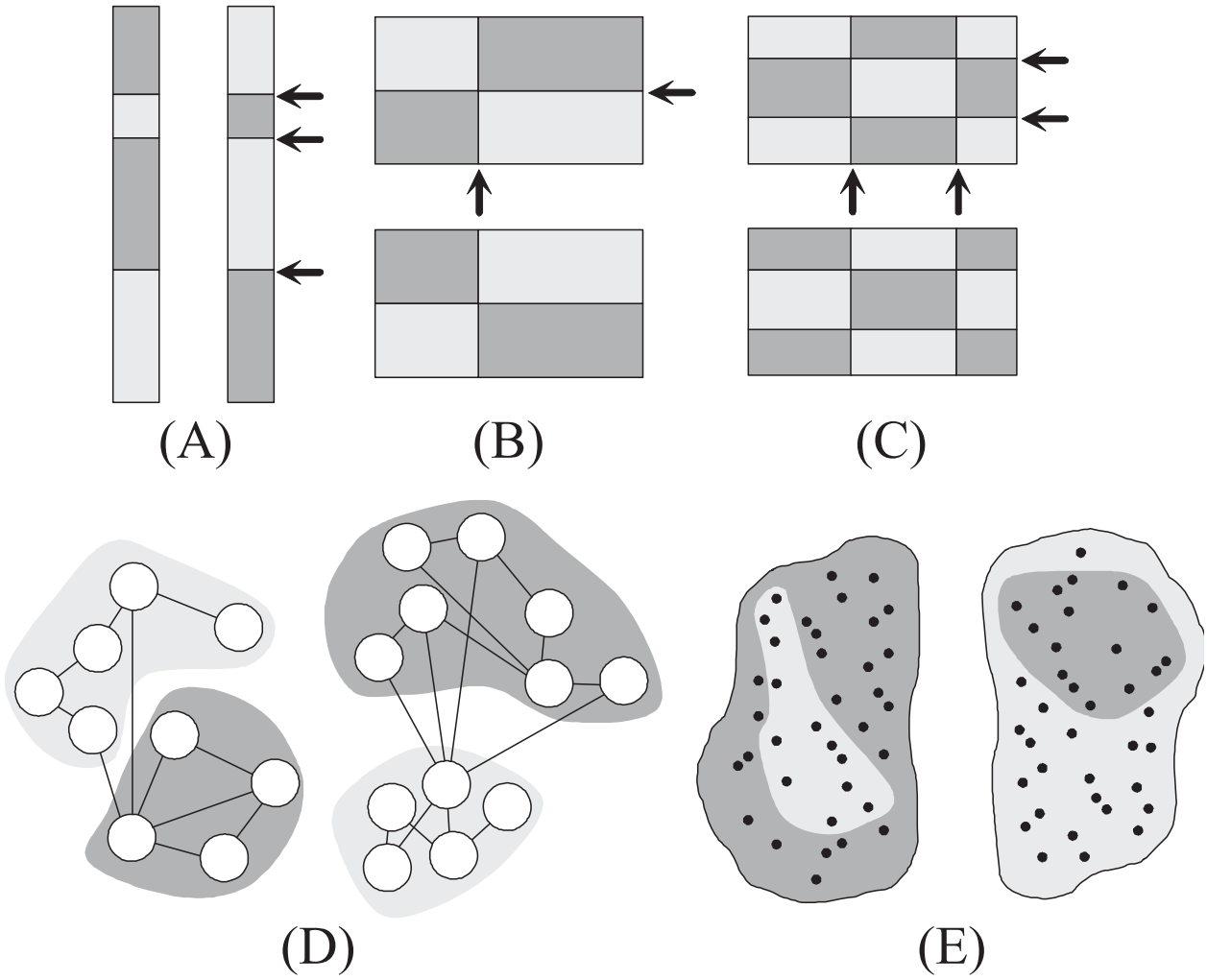
- The decoding of the i_{th} segment is given by:

$$\Delta\left[c(i,1),\cdots,c(i,l_0)\right] = x_{i,min} + \frac{x_{i,max} - x_{i,min}}{2^{l_0} - 1} \times \underbrace{\sum_{k=1}^{l_0} c(i, l_0 + 1 - k) \times 2^{k-1}}_{\in\left\{0,\, 2^{l_0} - 1\right\}}$$

- Here the resolution (and precision) of the search is regulated by the bit length l_0 and the ranges $[x_{i,min}, x_{i,max}]$. The distance between two successive grid points is:

$$\text{distance} = \frac{x_{i,max} - x_{i,min}}{2^{l_0} - 1}$$

Note, that the **shape** of chromosomes does not need to be string-based. 2D (or multidimensional) arrays, trees, graphs, sets or any other composite data-structure containing organised hierarchies of sub-structures can be used. For example:



Chromosome encodings and possible crossover scenarios. In all cases, offspring are generated by combining the parental parts of the similar shading.

(A): *3*-point linear crossover.

(B): One-line crossover on 2D-chromosomes.

(C): Two-line 2D crossover.

(D): Graph-based crossover.

(E): Set-based crossover.

Crossover

- This operator is important for **exploitation** of the solution space (i.e., exploit the current solution fragments or characteristics contained within the population).

- Many parental pairs are picked at random (but ones with high fitness are selected more often) to form pairs.

- These pairs then exchange some of their genetic information, to create new children.

- New child strings are created by randomly selecting a position on the parent strings (usually taken from uniform distribution) and exchanging each gene to one side of that position. This is called **crossover**.

Example:
The parents:
parent 1: 001000|0010
parent 2: 001100|1100
"could" produce:
child 1: 0010001100
child 2: 0011000010

- Of course, exchange of genetic information between identical strings does not change either string, and too many duplicates of a few strings can cause a loss of genetic diversity and premature convergence of the GA.

Mutation

- This operator is important for **exploration** of the solution space (explore new regions of the solution space not contained in the current population gene pool).
- As with any optimisation algorithm, there is often a tendency for a genetic algorithm to find solutions that are good, but not optimal.
  - The GA will reward a very good solution ahead of a mediocre one. This will give it an advantage when reproducing, but there is still no guarantee that this very good solution will be anything like the true optimal one.
  - If a population is dominated by a few strings, then there may be a very small chance that the string corresponding to the true solution will be generated by crossover alone.
- Mutation is usually applied after crossover, and it is intended to **prevent premature convergence and loss of genetic diversity**.
- Mutation introduces a small probability for each gene in a string to change spontaneously (1 becomes 0, etc.).

Example:
The children:
child 1: 0010001100
child 2: 0011000010
"could" become:
mutated child 1: 0010101100
mutated child 2: 0001000010

<END of Chapter 2>